

Introduction

Some LeetCode problems test whether you know an algorithm. Others test whether you can resist reaching for one. Construct Uniform Parity Array I belongs firmly in the second camp, and that's exactly why it's worth slowing down on.

At first glance, this problem looks like it wants graph theory, GCD reasoning, or some clever bit manipulation. The name alone — "parity array," "construct" — sounds like it's baiting you into overthinking. Interviewers love problems like this because they separate candidates who pattern-match against known algorithms from candidates who actually reason from first principles. If you can spot that this problem reduces to a single fact about subtraction and remainders, you'll write a three-line solution. If you can't, you'll spend fifteen minutes building machinery you never needed.

Learning to recognize when a problem is smaller than it looks is a skill in itself, and it transfers to nearly every technical interview you'll ever sit through.

Problem Overview

You're given an array of distinct integers. Your job is to build a second array of the same length where every single element ends up with the same parity — either every element is even, or every element is odd. Mixing is not allowed.

For each position in the new array, you have exactly two options:

1. Keep the original number exactly as it is.
2. Replace it with the difference between it and some other number from the original array.

There is no third option. No multiplying, no dividing, no adding. Just "keep" or "subtract one array element from another."

You need to return `True` if it's possible to build such a uniform array, and `False` otherwise — although, as we'll see, that second outcome never actually happens.

Constraints worth noting: the array has at most 100 elements, values range from 1 to 100, and all values are distinct. The distinctness matters because it guarantees that when you subtract one element from another, you're not subtracting a number from itself and trivially landing on zero.

Example

Take `nums = [2, 3]`.

- Position 0: subtract — $2 - 3 = -1$, which is odd.
- Position 1: keep as-is — `3`, which is odd.

Every slot lands on odd. Result: `True`.

Now take `nums = [4, 6]`. Both numbers are already even, so you keep both exactly as they are. Zero operations needed. Result: `True`.

Try `nums = [4, 7]` and imagine you only allowed yourself to keep values as-is. `4` is even, `7` is odd — that naive attempt fails immediately. This is the case that makes people think the answer sometimes has to be `False`. It doesn't, because you're allowed to subtract, and that extra move is enough to rescue every mixed case.

Intuition

Here's how an experienced engineer approaches this, rather than jumping straight to code.

Start by asking: what are the only ways this problem can go wrong? An all-even or all-odd array is impossible to build only if some element simply cannot become one of those two parities using our two allowed moves. So the real question is: can every number, using "keep" or "subtract another array value," always be pushed into at least one consistent parity across the whole array?

Break the array into two cases.

Case one: the array is already uniform. If every number is already even, keep them all as-is — you're done in zero moves. Same logic for all-odd. This case is trivial and costs nothing.

Case two: the array has a mix of even and odd numbers. This is the case that feels dangerous, and it's exactly where people start reaching for GCD arguments or bitwise tricks, because subtraction combined with number theory smells like it should be complicated. But strip away the noise: parity is arithmetic mod 2, and subtraction mod 2 behaves exactly like addition mod 2. Even minus odd, mod 2, is $0 - 1 = -1 \equiv 1 \pmod{2}$ — always odd. Always. It doesn't matter which even number or which odd number you pick, or how many of each exist in the array.

So if the array has at least one even and at least one odd number, every even element can subtract any odd element and land on odd, and every odd element already sits on odd. The whole array can be driven to all-odd in a single pass, regardless of how the values are distributed.

Combine both cases: uniform arrays are already done, and mixed arrays can always be forced to all-odd. There is no array configuration that fails. The function can simply return `True` unconditionally.

Brute Force Solution

Idea: Check if the array is already all-even or all-odd. If not, attempt every combination of keep/subtract per element and see if any combination produces a uniform-parity array.

Algorithm: 1. Check `all(x % 2 == 0 for x in nums)` and `all(x % 2 == 1 for x in nums)`.
2. If neither holds, try, for each element, either keeping it or subtracting every other array element from it, and check if any full assignment produces uniform parity.

Advantages: Conceptually simple, doesn't require the parity insight.

Disadvantages: Wildly unnecessary — you'd be exploring a combinatorial space to answer a question that's actually always `True`. It also obscures the real insight instead of revealing it.

Complexity: In the worst case, trying every keep/subtract choice across every element against every other element is $O(n^2)$ per configuration and exponential if you're checking assignments naively. Completely avoidable.

Optimal Solution

The optimal solution is really just applying the intuition above directly, with no loop required.

Array state	Move to use	Result
All even	Keep every element	All even
All odd	Keep every element	All odd
Mixed even/odd	Odd elements: keep. Even elements: subtract any odd element	All odd

Since every branch of that table lands on `True`, there's no need to even check which branch you're in. The function collapses to a single unconditional return.

Python Code

```
class Solution:
    def canConstruct(self, nums1: list[int]) -> bool:
        # Uniform arrays need no change; mixed arrays can always be
        # driven to all-odd since even - odd is always odd (mod 2).
        return True
```

Code Walkthrough

- `class Solution` and `canConstruct` match LeetCode's required interface for this problem.
- `nums1` is accepted as the input array, per the problem signature, though its contents never change the outcome.
- The comment documents the two cases that justify the constant return: an already-uniform array needs no moves, and a mixed array can always be pushed to all-odd because subtracting an odd value from an even value is always odd under modular arithmetic.
- The function returns `True` unconditionally because no input configuration under the given constraints can fail.

Dry Run

Take `nums1 = [2, 3]` .

1. Call `canConstruct([2, 3])` .
2. The function does not inspect the array at all — it returns `True` immediately.
3. Cross-check by hand: position 0 uses subtract ($2 - 3 = -1$, odd), position 1 keeps `3` (odd). Both slots land on odd, confirming `True` was the correct answer.

Take `nums1 = [4, 8, 16]` .

1. Call `canConstruct([4, 8, 16])` .
2. Returns `True` immediately.
3. Cross-check: all three values are already even, so keeping every element as-is produces an all-even array with zero moves. Confirmed.

Complexity Analysis

Time complexity: $O(1)$. There is no loop over the array and no dependency on `n` , the array length. The function returns a constant value regardless of input size.

Space complexity: $O(1)$. No auxiliary data structures are allocated.

These are correct — not just fast — because the underlying math guarantees the answer is always `True` for every valid input under the problem's constraints (distinct integers between 1 and 100). There's no input that requires the function to do work to determine the answer.

Alternative Solutions

A reasonable alternative is to explicitly branch on the array's current state before returning `True`, for readability in an interview setting:

```
class Solution:
    def canConstruct(self, nums1: list[int]) -> bool:
        if all(x % 2 == 0 for x in nums1):
            return True
        if all(x % 2 == 1 for x in nums1):
            return True
        # mixed parity: even - odd is always odd
        return True
```

This is functionally identical but $O(n)$ instead of $O(1)$, since it scans the array with `all()`. It's worth mentioning in an interview to show you considered the input, even though the unconditional `True` is the leaner, more honest solution once you've proven the result never depends on the array's contents.

Edge Cases

- **Single-element array:** Trivially uniform already — keep the one element as-is. Still `True`.
- **All values identical parity, all even:** Keep every element. `True`.
- **All values identical parity, all odd:** Keep every element. `True`.
- **Maximum size array (100 elements) with maximum spread of values (1 to 100):** Still `True` — array size and value range never affect the outcome.
- **Array with exactly one even and one odd value:** The even value subtracts the odd value to become odd; the odd value stays. `True`.

Note: since the array elements are guaranteed distinct, there's no case where subtracting a number from itself produces zero-related ambiguity.

Common Mistakes

1. **Assuming "keep as-is" is the only allowed move and concluding mixed arrays fail.** The subtract option is what makes the answer always `True`, and skipping it leads directly to a wrong `False`.
2. **Reaching for GCD-based reasoning.** This problem has nothing to do with divisibility chains; it's pure parity arithmetic.

3. **Overcomplicating with bitwise tricks.** Checking the lowest bit of every possible subtraction result is unnecessary once you know even minus odd is always odd.
4. **Writing an $O(n)$ or worse solution when $O(1)$ suffices.** Looping to check `all(even)` or `all(odd)` isn't wrong, but it signals you haven't fully collapsed the problem to its core truth.
5. **Forgetting to consider the mixed case entirely.** Some solvers correctly handle "already uniform" but panic on mixed arrays and assume more logic is needed.
6. **Not verifying the parity fact algebraically.** Assuming even minus odd "feels" odd is different from confirming it via mod-2 arithmetic — always confirm intuition with the actual math before trusting it.

Interview Questions

- Why is subtraction mod 2 equivalent to addition mod 2?
- Would this problem change if the allowed operation were addition instead of subtraction?
- What if the array could contain duplicate values — does the "distinct" constraint matter here, and why?
- How would you handle a variant that asks for the minimum number of operations rather than just feasibility?
- Could you write a test suite that convinces a skeptical teammate this function is correct without enumerating every array?

Similar Problems

- **LeetCode: Minimum Number of Operations to Make Array Continuous** — another problem where the naive brute-force instinct is far more expensive than the actual pattern hiding underneath.
- **LeetCode: Check if Array Is Sorted and Rotated** — tests the same skill of recognizing when a property can be verified with a single pass instead of complex logic.
- **LeetCode: Divide Array Into Equal Pairs** — another parity-driven problem where counting even/odd occurrences resolves the whole question.
- **LeetCode: Construct Uniform Parity Array II** (if applicable in your problem set) — the natural follow-up that likely asks for the minimum operations rather than a yes/no answer, forcing you to actually count even-to-odd conversions.

Key Takeaways

- Not every problem needs an algorithm — some need a single correct observation.

- Parity arithmetic (mod 2) is often simpler than it looks: even minus odd is always odd, no exceptions.
- Before writing code, enumerate every move you're allowed to make and ask what each one guarantees. That's often faster than testing case after case.
- The best solutions are sometimes the shortest, and proving correctness matters more than writing more code.

Watch the Video

Prefer to see this traced out visually, with the quiz checkpoints and the full walkthrough? Watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series dedicated to making LeetCode problems, coding interview prep, and core software engineering concepts genuinely intuitive — not just memorized. Each video and article pairs a real problem with the reasoning an experienced engineer would actually use to solve it, so the goal isn't just "here's the answer," it's "here's how you'd find it yourself next time."

Call To Action

If this breakdown saved you from overthinking a problem that didn't need it, consider liking the video on YouTube and subscribing for more daily LeetCode walkthroughs. For deeper dives and written companions to every video, subscribe to the Substack newsletter. Got a different way you solved this one, or a problem you want covered next? Drop it in the comments — and if this helped, share it with someone prepping for interviews.

Quick Review

Parity-flip pattern: when to think 'toggle LSB in pairs'?

When array must become all-odd/all-even and only +1/-1 moves allowed: pair mismatched elements, flip one bit each.

Time complexity of the uniform-parity construction?

$O(n)$ — single pass to count/split odds and evens, one more pass to apply moves.

Space complexity of the uniform-parity solution?

$O(1)$ extra ($O(n)$ for output array) — just counters and in-place adjustment, no auxiliary structures.

Trickiest edge case in Construct Uniform Parity Array I?

Array already uniform parity — must return unchanged array with zero moves, not force unnecessary flips.

Why must you flip toward majority parity, not minority?

Constraints cap total moves; converting the smaller group costs fewer $+1/-1$ ops than flipping the larger group.

Interview follow-up: what changes in a 'Uniform Parity II' with a move-count limit?

Becomes a greedy/counting feasibility check — verify $\min(\text{odds}, \text{evens}) \text{ flips} \leq \text{limit}$ before committing to majority parity.